적절한 출처 표기가 있는 경우, Google은 본 논문의 표와 그림을 저널리즘 또는 학술 목적으로만 복제할 수 있도록 허용한다.

어텐션만으로 충분하다 (Attention Is All You Need)

Ashish Vaswani* · Google Brain · avaswani@google.com
Noam Shazeer* · Google Brain · noam@google.com
Niki Parmar* · Google Research · nikip@google.com
Jakob Uszkoreit* · Google Research · usz@google.com
Llion Jones* · Google Research · llion@google.com
Aidan N. Gomez*† · University of Toronto · aidan@cs.toronto.edu
Łukasz Kaiser* · Google Brain · lukaszkaiser@google.com
Illia Polosukhin*‡ · illia.polosukhin@gmail.com

초록

지배적인 순열 변환(sequence transduction) 모델은 인코더와 디코더를 포함한 복잡한 순환 신경망이나 합성곱 신경망에 기반한다. 성능이 가장 좋은 모델은 또한 인코더와 디코더를 어텐션(attention) 메커니즘으로 연결한다. 본 논문은 순환과 합성곱을 전부 배제하고 오직 어텐션 메커니즘만에 기반한 새로운 단순 네트워크 구조인 트랜스포머(Transformer)를 제안한다. 두 가지 기계 번역 과제에 대한 실험에서, 제안 모델은 품질 면에서 우수하면서 병렬화 가능성이 더 크고 학습 시간이 현저히 짧음을 보여 준다. WMT 2014 영어-독일어 번역 과제에서 본 모델은 BLEU 28.4를 달성하여, 앙상블을 포함한 기존 최고 결과보다 2 BLEU 이상 향상시켰다. WMT 2014 영어-프랑스어 번역 과제에서는 8개의 GPU로 3.5일 학습한 뒤 단일 모델 BLEU 41.8이라는 새로운 최첨단 점수를 기록했으며, 이는 문헌의 최고 모델들에 비해 학습 비용의 극히 일부에 해당한다. 넓은 학습 데이터와 제한된 학습 데이터 모두에서 영어 구문 분석(constituency parsing)에 성공적으로 적용함으로써, 트랜스포머가 다른 과제에도 잘 일반화됨을 보였다.

*기여 동등. 저자 순서는 무작위이다. Jakob은 RNN을 자기-어텐션으로 바꾸자고 제안하고 이 아이디어를 평가하려는 노력을 시작했다. Ashish는 Illia와 함께 최초의 트랜스포머 모델을 설계·구현했으며 이 연구의 모든 측면에 핵심적으로 관여했다. Noam은 스케일된 내적 어텐션, 다중 헤드 어텐션, 매개변수 없는 위치 표현을 제안했으며 거의 모든 세부에 함께 관여한 또 다른 사람이 되었다. Niki는 원본 코드베이스와 tensor2tensor에서 수많은 모델 변형을 설계·구현·튜닝·평가했다. Llion은 새로운 모델 변형 실험, 초기 코드베이스, 효율적 추론과 시각화를 담당했다. Łukasz와 Aidan은 tensor2tensor의 여러 부분 설계와 구현, 이전 코드베이스 대체, 결과 대폭 개선과 연구 가속에 수많은 날을 보냈다.

†본 연구는 Google Brain 재직 시 수행.

‡본 연구는 Google Research 재직 시 수행.

제31회 신경정보처리시스템학회(NIPS 2017), 롱비치, CA, 미국.

1. 서론

순환 신경망, 특히 장단기 기억(long short-term memory) [13]과 게이트 순환(gated recurrent) 신경망 [7]은 언어 모델링과 기계 번역 등 순열 모델링 및 변환 문제에서 최첨단 접근으로 확고히 자리 잡았다 [35, 2, 5]. 그 이후에도 순환 언어 모델과 인코더-디코더 아키텍처의 한계를 넓히려는 노력이 이어졌다 [38, 24, 15].

순환 모델은 일반적으로 입력·출력 순열의 기호 위치를 따라 계산을 인수분해한다. 위치를 계산 시간의 단계에 맞추면서, 각 단계에서 이전 은닉 상태 h_{t-1}과 위치 t의 입력에 대한 함수로 은닉 상태 h_t의 순열을 생성한다. 이러한 본질적으로 순차적인 특성 때문에 학습 예제 내 병렬화가 막히며, 시퀀스가 길어질수록 중요해진다. 메모리 제약이 예제 간 배치를 제한하기 때문이다. 최근 연구는 인수분해 트릭과 조건부 계산을 통해 계산 효율을 크게 높였고, 후자의 경우 모델 성능도 개선했다. 그러나 순차 계산의 근본적 제약은 남아 있다.

어텐션 메커니즘은 다양한 과제에서 입력·출력 순열에서의 거리와 무관하게 의존성을 모델링할 수 있게 하여 매력적인 순열 모델의 필수 요소가 되었다 [2, 19]. 그러나 소수의 경우 [27]을 제외하고는 이러한 어텐션은 순환 네트워크와 함께 쓰인다.

본 연구에서는 순환을 버리고 입력과 출력 사이의 전역 의존성을 전적으로 어텐션 메커니즘으로 그리는 모델 구조, 즉 트랜스포머를 제안한다. 트랜스포머는 훨씬 더 많은 병렬화를 허용하며, 8개의 P100 GPU에서 12시간만 학습해도 번역 품질 면에서 새로운 최첨단에 도달할 수 있다.

2. 배경

순차 계산을 줄이려는 목표는 확장 신경 GPU(Extended Neural GPU), ByteNet, ConvS2S의 기반이 되기도 하며, 이들은 모두 합성곱 신경망을 기본 블록으로 사용하여 모든 입·출력 위치에 대해 은닉 표현을 병렬로 계산한다. 이 모델들에서 임의의 두 입·출력 위치에서 신호를 연결하는 데 필요한 연산 수는 위치 간 거리에 대해 ConvS2S에서는 선형, ByteNet에서는 로그적으로 증가한다. 이는 먼 위치 간 의존성 학습을 어렵게 만든다 [12]. 트랜스포머에서는 이것이 상수 번의 연산으로 줄어들지만, 어텐션 가중 위치의 평균으로 인한 유효 해상도 감소라는 비용이 있으며, 3.2절에서 기술하는 다중 헤드 어텐션으로 이를 완화한다.

자기-어텐션(self-attention)은 때로 내부-어텐션(intra-attention)이라 불리며, 단일 순열의 서로 다른 위치들을 연결하여 순열의 표현을 계산하는 어텐션 메커니즘이다. 자기-어텐션은 독해, 요약, 함의, 과제 독립 문장 표현 학습 등 여러 과제에서 성공적으로 쓰였다 [4, 27, 28, 22].

종단간 메모리 네트워크는 순열 정렬 순환 대신 순환 어텐션 메커니즘에 기반하며, 단순 언어 질의응답과 언어 모델링 과제에서 좋은 성능을 보였다 [34].

우리가 아는 한, 트랜스포머는 순열 정렬 RNN이나 합성곱 없이 오직 자기-어텐션만으로 입·출력 표현을 계산하는 최초의 변환(transduction) 모델이다. 이후 절에서는 트랜스포머를 기술하고, 자기-어텐션을 정당화하며, [17, 18] 및 [9]와 같은 모델과 비교하여 장점을 논한다.

3. 모델 아키텍처

대부분의 경쟁적 신경 순열 변환 모델은 인코더-디코더 구조를 갖는다 [5, 2, 35]. 여기서 인코더는 기호 표현의 입력 순열 $(x_1, ..., x_n)$을 연속 표현 순열 $z = (z_1, ..., z_n)$으로 사상한다. z가 주어지면 디코더는 한 번에 하나씩 출력 순열 $(y_1, ..., y_m)$을 생성한다. 각 단계에서 모델은 자기회귀(auto-regressive) [10]로, 다음 기호를 생성할 때 이전에 생성된 기호들을 추가 입력으로 사용한다.

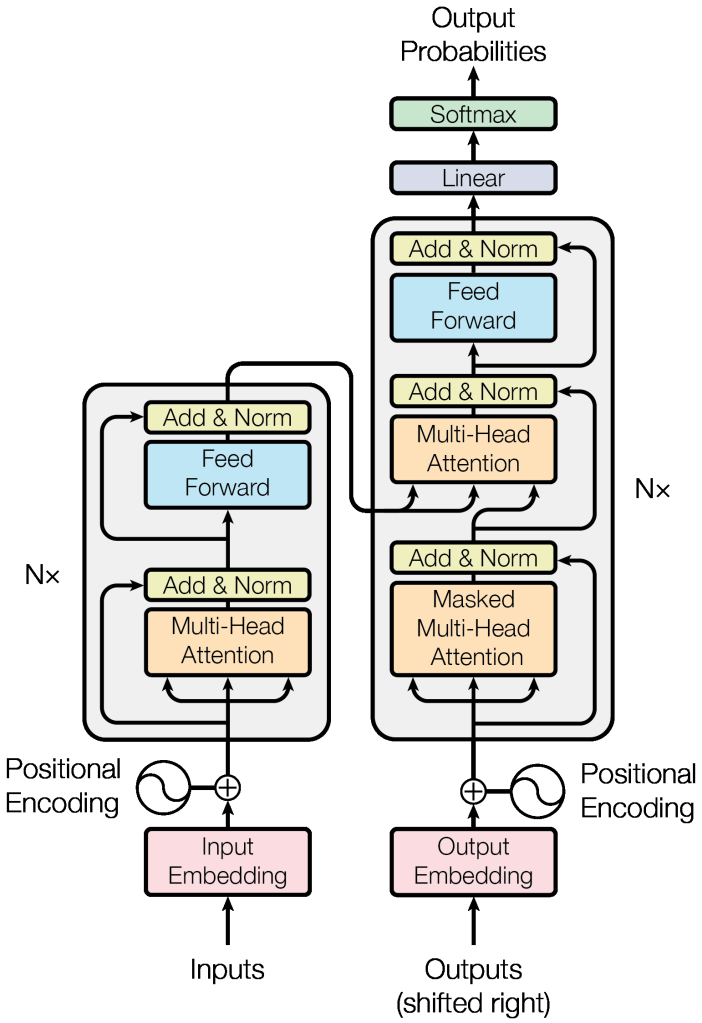

그림 1: 그림 1: 트랜스포머 — 모델 아키텍처.

트랜스포머는 이 전체 구조를 따르며, 인코더와 디코더 모두에 쌓인 자기-어텐션과 위치별(point-wise) 전결합 층을 사용한다. 그림 1의 왼쪽·오른쪽 절반에 각각 나타나 있다.

3.1. 인코더와 디코더 스택

인코더: 인코더는 $N = 6$개의 동일 층을 쌓은 것으로 구성된다. 각 층에는 두 개의 하위층이 있다. 첫째는 다중 헤드 자기-어텐션, 둘째는 위치별 전결합 피드포워드 네트워크이다. 두 하위층 모두에 잔차 연결(residual connection) [11]을 두고, 그다음 층 정규화(layer normalization) [1]를 적용한다. 즉 각 하위층의 출력은 $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$이며, $\mathrm{Sublayer}(x)$는 해당 하위층이 구현하는 함수이다. 잔차 연결을 위해 모델의 모든 하위층과 임베딩 층의 출력 차원은 $d_{\mathrm{model}} = 512$로 맞춘다.

디코더: 디코더 역시 $N = 6$개의 동일 층으로 구성된다. 인코더 층의 두 하위층에 더해, 디코더는 인코더 스택의 출력에 대한 다중 헤드 어텐션을 수행하는 세 번째 하위층을 삽입한다. 인코더와

마찬가지로 각 하위층에 잔차 연결 후 층 정규화를 적용한다. 또한 디코더 스택의 자기-어텐션 하위층을 수정하여 이후 위치를 보지 못하게 한다. 이 마스킹과 출력 임베딩이 한 위치만큼 이동한다는 사실이 결합되어, 위치 i에 대한 예측은 i보다 작은 위치에서 알려진 출력에만 의존하도록 한다.

3.2. 어텐션

어텐션 함수는 질의(query)와 키-값(key-value) 쌍들의 집합을 출력으로 사상하며, 질의·키·값·출력은 모두 벡터이다. 출력은 값들의 가중합으로 계산되며, 각 값에 부여되는 가중치는 질의와 해당 키와의 호환 함수로부터 계산된다.

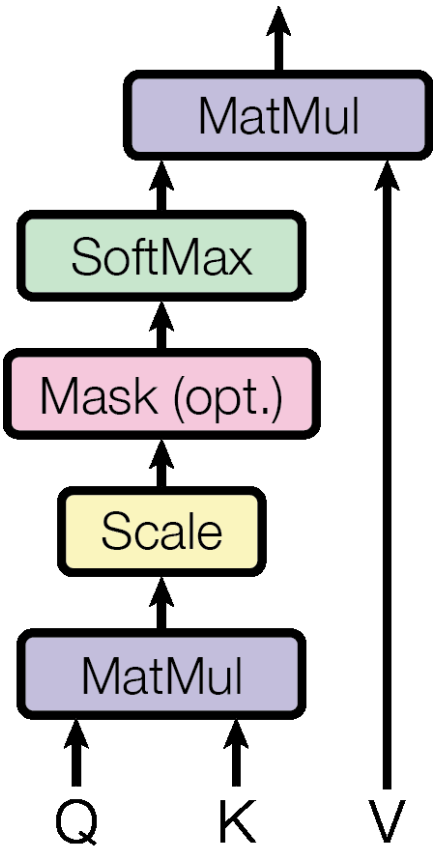

그림 2: (왼쪽) 스케일된 내적 어텐션.

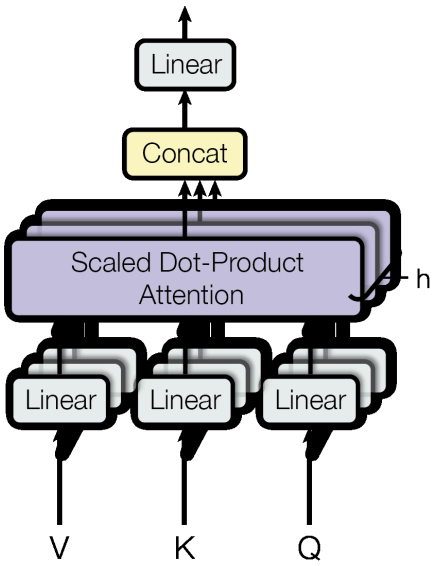

그림 3: (오른쪽) 다중 헤드 어텐션은 여러 어텐션 층이 병렬로 동작한다.

3.2.1. 스케일된 내적 어텐션

우리가 사용하는 어텐션은 "스케일된 내적 어텐션"(그림 2)이라 부른다. 입력은 차원 d_k의 질의·키와 차원 d_v의 값으로 구성된다. 질의와 모든 키의 내적을 계산하고, 각 값을 $\sqrt{d_k}$로 나눈 뒤, 소프트맥스를 적용하여 값들에 대한 가중치를 얻는다.

실제로는 질의들을 행렬 Q로 묶어 한꺼번에 계산하며, 키와 값도 K, V로 묶는다. 출력 행렬은 다음과 같다.

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{Q\,K^T}{\sqrt{d_k}}\right) V$$

가장 널리 쓰이는 두 어텐션은 가산 어텐션(additive attention) [2]과 내적(곱) 어텐션이다. 내적 어텐션은 스케일 인자 $\frac{1}{\sqrt{d_k}}$를 제외하면 우리 알고리즘과 같다. 가산 어텐션은 단일 은닉층을 가진 피드포워드 네트워크로 호환 함수를 계산한다. 이론적 복잡도는 비슷하지만, 내적 어텐션은 고도로 최적화된 행렬 곱 코드로 구현할 수 있어 실제로 훨씬 빠르고 메모리 효율적이다.

d_k가 작을 때는 두 메커니즘이 비슷한 성능을 내지만, d_k가 커지면 스케일 없는 내적 어텐션보다 가산 어텐션이 우수하다 [3]. d_k가 크면 내적의 크기가 커져 소프트맥스가 기울기가 극도로 작은 영역으로 밀려 난다고 추정한다[1]. 이를 완화하기 위해 내적을 $\frac{1}{\sqrt{d_k}}$로 스케일한다.

3.2.2. 다중 헤드 어텐션

d_{model}차원의 키·값·질의에 대해 단일 어텐션을 쓰는 대신, 서로 다른 학습된 선형 사영으로 질의·키·값을 h번 d_k, d_k, d_v차원으로 사영하는 편이 유리함을 확인했다. 사영된 질의·키·값 각각에 대해 어텐션을 병렬로 적용하면 d_v차원 출력이 나오며, 이를 이어 붙인 뒤 다시 사영하면 최종 값이 된다 (그림 2).

다중 헤드 어텐션은 서로 다른 위치에서 서로 다른 표현 부분공간의 정보에 동시에 주목할 수 있게 한다. 단일 헤드에서는 평균화가 이를 억제한다.

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, ..., \mathrm{head}_h)W^O$$

여기서 $\mathrm{head}_i = \mathrm{Attention}\left(Q\ W_i^Q, K\ W_i^K, V\ W_i^V\right)$이며, 사영 행렬은 $W_i^Q \in R^{d_{model} \times d_k}$, $W_i^K \in R^{d_{model} \times d_k}$, $W_i^V \in R^{d_{model} \times d_v}$, $W^O \in R^{h\ d_v \times d_{model}}$이다.

본 연구에서는 $h = 8$개의 병렬 어텐션 층(헤드)을 사용하며, 각 헤드에 $d_k = d_v = \frac{d_{model}}{h} = 64$를 쓴다. 헤드당 차원이 줄어들어, 전체 계산 비용은 전 차원 단일 헤드 어텐션과 유사하다.

3.2.3. 모델에서의 어텐션의 사용

트랜스포머는 다중 헤드 어텐션을 세 가지 방식으로 쓴다:

- **인코더-디코더 어텐션** 층에서는 질의는 이전 디코더 층에서 오고, 키와 값은 인코더 출력에서 온다. 이를 통해 디코더의 모든 위치가 입력 순열의 모든 위치에 주목할 수 있다. 전형적인 시퀀스-투-시퀀스 모델의 인코더-디코더 어텐션과 같다 [38, 2, 9].
- **인코더**에는 자기-어텐션 층이 있다. 자기-어텐션에서는 키·값·질의가 모두 같은 출처, 즉 인코더의 이전 층 출력에서 온다. 인코더의 각 위치는 이전 층의 모든 위치에 주목할 수 있다.
- **디코더**의 자기-어텐션도 마찬가지로 디코더의 해당 위치까지의 모든 위치에 주목할 수 있다. 자기회귀 성질을 지키려면 디코더에서 왼쪽으로의 정보 흐름을 막아야 하며, 이는 스케일된 내적 어텐션 내부에서 소프트맥스 입력 중 불법 연결에 해당하는 값을 $-\infty$로 마스킹하여 구현한다 (그림 2).

3.3. 위치별 피드포워드 네트워크

어텐션 하위층 외에도 인코더와 디코더의 각 층에는 전결합 피드포워드 네트워크가 있으며, 위치마다 따로·동일하게 적용된다. ReLU 활성화를 끼운 두 선형 변환으로 구성된다.

[1] 내적이 커지는 이유를 보이기 위해, q와 k의 성분이 평균 0·분산 1인 독립 확률변수라고 가정하자. 그러면 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$는 평균 0·분산 d_k를 갖는다.

$$\mathrm{FFN(x) = max(0, x\ W_1 + b_1)W_2 + b_2}$$

선형 변환은 위치마다 같지만 층마다 다른 매개변수를 쓴다. 커널 크기 1인 합성곱 두 번으로도 기술할 수 있다. 입·출력 차원은 $\mathrm{d_{model} = 512}$, 내부층은 $\mathrm{d_{ff} = 2048}$이다.

3.4. 임베딩과 소프트맥스

다른 순열 변환 모델과 마찬가지로 학습된 임베딩으로 입·출력 토큰을 $\mathrm{d_{model}}$차원 벡터로 바꾼다. 디코더 출력을 다음 토큰 확률로 바꾸기 위해 통상의 학습된 선형 변환과 소프트맥스를 쓴다. 두 임베딩 층과 소프트맥스 직전 선형 변환 사이에 동일한 가중 행렬을 공유한다 [30]. 임베딩 층에서는 해당 가중치에 $\sqrt{\mathrm{d_{model}}}$을 곱한다.

3.5. 위치 인코딩

모델에 순환이나 합성곱이 없으므로, 순열의 순서 정보를 주입해야 한다. 이를 위해 인코더·디코더 스택 하단의 입력 임베딩에 "위치 인코딩"을 더한다. 위치 인코딩은 임베딩과 같은 차원 $\mathrm{d_{model}}$이어서 더할 수 있다. 학습형·고정형 등 여러 선택이 있다 [9].

본 연구에서는 서로 다른 주파수의 사인·코사인 함수를 사용한다.

$$\mathrm{PE_{pos,\ 2i} = sin\left(\frac{pos}{10000^{2\frac{i}{d_{model}}}}\right)}$$

$$\mathrm{PE_{pos,\ 2i+1} = cos\left(\frac{pos}{10000^{2\frac{i}{d_{model}}}}\right)}$$

여기서 pos는 위치, i는 차원이다. 위치 인코딩의 각 차원은 사인파에 대응하며, 파장은 $2\ \pi$에서 $10000 \cdot 2\ \pi$까지 기하급수적으로 이어진다. 고정 오프셋 k에 대해 $\mathrm{PE_{pos+k}}$가 $\mathrm{PE_{pos}}$의 선형 함수로 표현될 수 있어 상대 위치에 쉽게 주목할 수 있으리라는 가설로 이 함수를 택했다.

학습된 위치 임베딩 [9]도 실험했으며, 두 버전은 거의 동일한 결과를 냈다(표 3의 행 (E)). 사인 버전을 선택한 이유는 학습에서 본 것보다 긴 순열로 외삽할 수 있기 때문이다.

표 1: 층 유형별 최대 경로 길이, 층당 복잡도, 순차 연산 최소 개수. n은 순열 길이, d는 표현 차원, k는 합성곱의 커널 크기, r은 제한 자기-어텐션의 이웃 반경이다.

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

4. 자기-어텐션을 택한 이유

이 절에서는 자기-어텐션 층을 기호 표현의 가변 길이 순열 $(\mathrm{x_1, ..., x_n})$을 동일 길이의 다른 순열 $(\mathrm{z_1, ..., z_n})$으로 사상하는 데 흔히 쓰이는 순환·합성곱 층과 여러 측면에서 비교한다. 여기서

$x_i, z_i \in R^d$이며, 전형적인 순열 변환 인코더나 디코더의 은닉 층에 해당한다. 자기-어텐션 사용을 정당화하기 위해 세 가지 바람직한 성질을 고려한다.

첫째는 층당 전체 계산 복잡도이다. 둘째는 병렬화할 수 있는 계산량으로, 필요한 순차 연산의 최소 개수로 잰다. 셋째는 네트워크에서 장거리 의존성 간의 경로 길이다. 장거리 의존성 학습은 많은 순열 변환 과제의 핵심 난제이다. 그러한 의존성을 학습하는 능력에 영향을 주는 요인 중 하나는 순방향·역방향 신호가 통과해야 하는 경로의 길이다. 입·출력 순열의 임의 위치 조합에 대해 이러한 경로가 짧을수록 장거리 의존성을 더 쉽게 학습한다 [12]. 따라서 서로 다른 층 유형으로 구성된 네트워크에서 임의 두 입·출력 위치 간 최대 경로 길이도 비교한다.

표 1에서 볼 수 있듯, 자기-어텐션 층은 상수 번의 순차 연산으로 모든 위치를 연결하는 반면, 순환 층은 $O(n)$번의 순차 연산이 필요하다. 계산 복잡도 측면에서, 순열 길이 n이 표현 차원 d보다 작을 때 자기-어텐션 층이 순환 층보다 빠른데, 기계 번역의 최첨단 모델이 쓰는 워드피스·바이트 쌍 표현처럼 문장 표현에서 흔히 그렇다. 매우 긴 순열을 다루는 과제의 계산 성능을 높이기 위해, 각 출력 위치 주변 크기 r의 이웃만 고려하도록 자기-어텐션을 제한할 수 있다. 그러면 최대 경로 길이는 $O(n/r)$로 늘어난다. 이 접근은 향후 연구에서 더 살펴볼 계획이다.

커널 폭 $k < n$인 단일 합성곱 층은 모든 입·출력 위치 쌍을 연결하지 못한다. 인접 커널의 경우 $O(n/k)$개의 합성곱 층 스택이, 팽창 합성곱의 경우 $O(\log_k n)$이 필요하며, 네트워크의 임의 두 위치 간 최장 경로가 길어진다. 합성곱 층은 일반적으로 순환 층보다 k배 비용이 더 든다. 그러나 분리 합성곱(separable convolution) [6]은 복잡도를 $O\left(k \cdot n \cdot d + n \cdot d^2\right)$까지 크게 줄인다. 설령 $k = n$이어도, 분리 합성곱의 복잡도는 자기-어텐션 층과 위치별 피드포워드 층의 조합과 같으며, 후자가 본 모델이 취하는 방식이다.

부수적으로 자기-어텐션은 더 해석 가능한 모델을 줄 수 있다. 부록에서 모델의 어텐션 분포를 살펴보고 예를 논한다. 개별 어텐션 헤드가 서로 다른 과제를 수행하는 것이 분명할 뿐 아니라, 많은 헤드가 문장의 통사·의미 구조와 관련된 행동을 보인다.

5. 학습

이 절에서는 모델의 학습 절차를 기술한다.

5.1. 학습 데이터와 배치

표준 WMT 2014 영어-독일어 데이터셋(약 450만 문장 쌍)으로 학습했다. 문장은 바이트 쌍 인코딩 [3]으로 부호화했으며, 공유 소스-타깃 어휘는 약 37,000 토큰이다. 영어-프랑스어는 훨씬 큰 WMT 2014 영어-프랑스어 데이터셋(36M 문장)을 쓰고, 32,000 워드피스 어휘 [38]로 토큰을 나눴다. 문장 쌍은 근사 시퀀스 길이별로 묶었다. 각 학습 배치는 약 25,000개의 소스 토큰과 25,000개의 타깃 토큰을 포함하는 문장 쌍 집합이었다.

5.2. 하드웨어와 일정

8개의 NVIDIA P100 GPU가 장착된 한 대의 머신에서 학습했다. 본문에 기술한 하이퍼파라미터를 쓰는 베이스 모델에서 학습 스텝당 약 0.4초가 걸렸다. 베이스 모델은 총 100,000 스텝(12시간) 학습했다. 빅 모델(표 3 맨 아래 줄)은 스텝당 1.0초였고, 300,000 스텝(3.5일) 학습했다.

5.3. 옵티마이저

Adam 옵티마이저 [20]에 $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\varepsilon = 10^{-9}$를 사용했다. 학습률은 다음 식에 따라 스텝에 따라 바꿨다.

$$\mathrm{lrate} = \mathrm{d}_{\mathrm{model}}^{-0.5} \cdot \min\left(\mathrm{step_num}^{-0.5}, \mathrm{step_num} \cdot \mathrm{warmup_steps}^{-1.5}\right)$$

처음 $\mathrm{warmup_steps}$ 스텝 동안은 학습률을 선형으로 올리고, 이후에는 스텝 번호의 역제곱근에 비례해 내린다. $\mathrm{warmup_steps} = 4000$을 사용했다.

5.4. 정규화

학습 중 세 가지 정규화를 쓴다.

잔차 드롭아웃(residual dropout): 각 하위층 출력에 드롭아웃 [33]을 적용한 뒤 하위층 입력과 더하고 정규화한다. 또한 인코더·디코더 스택에서 임베딩과 위치 인코딩의 합에도 드롭아웃을 적용한다. 베이스 모델에서 $\mathrm{P}_{\mathrm{drop}} = 0.1$을 썼다.

레이블 스무딩: 학습 시 레이블 스무딩 $\varepsilon_{\mathrm{ls}} = 0.1$ [36]을 사용했다. 모델이 더 불확실해지므로 퍼플렉서티에는 불리하지만, 정확도와 BLEU는 개선된다.

표 2: 트랜스포머는 영어-독일어 및 영어-프랑스어 newstest2014에서 이전 최첨단 모델보다 더 나은 BLEU 점수를 학습 비용의 극히 일부로 달성한다.

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

6. 결과

6.1. 기계 번역

WMT 2014 영어-독일어 번역 과제에서, 빅 트랜스포머 모델(표 2의 Transformer (big))은 이전에 보고된 최고 모델(앙상블 포함)보다 2.0 BLEU 이상 앞서며 BLEU 28.4의 새로운 최첨단을 세웠다. 이

모델의 구성은 표 3 맨 아래 줄에 나열되어 있다. 학습은 P100 GPU 8대로 3.5일이 걸렸다. 베이스 모델만으로도 경쟁 모델들의 학습 비용의 일부로 모든 이전 단일 모델과 앙상블을 능가한다.

WMT 2014 영어-프랑스어 번역 과제에서 빅 모델은 BLEU 41.0을 달성하여, 이전 단일 모델 최첨단보다 학습 비용이 1/4 미만임에도 모든 이전 단일 모델을 앞질렀다. 영어-프랑스어용 빅 트랜스포머는 드롭아웃 $\mathrm{P}_{\mathrm{drop}} = 0.3$ 대신 0.1을 썼다.

베이스 모델은 10분 간격으로 저장된 마지막 5개 체크포인트를 평균한 단일 모델을 썼다. 빅 모델은 마지막 20개를 평균했다. 빔 크기 4, 길이 패널티 $\alpha = 0.6$인 빔 탐색 [38]을 사용했다. 이러한 하이퍼파라미터는 개발 집합에서 실험해 고정했다. 추론 시 최대 출력 길이는 입력 길이 + 50으로 두되, 가능하면 일찍 종료했다 [38].

표 2는 결과를 요약하고 번역 품질·학습 비용을 문헌의 다른 아키텍처와 비교한다. 학습에 사용한 부동소수점 연산 수는 학습 시간, GPU 개수, GPU당 지속 단정밀 부동소수점 처리량 추정의 곱으로 추정했다[2].

6.2. 모델 변형

트랜스포머 구성 요소의 중요도를 평가하기 위해 베이스 모델을 여러 방식으로 바꾸고, 영어-독일어 번역의 newstest2013 개발 집합에서 성능 변화를 쟀다. 이전 절과 같이 빔 탐색을 썼으나 체크포인트 평균은 하지 않았다. 결과는 표 3에 있다.

표 3: 트랜스포머 아키텍처의 변형. 나열되지 않은 값은 베이스 모델과 동일하다. 모든 지표는 영어-독일어 번역 개발 집합 newstest2013 기준이며, 퍼플렉서티는 바이트 쌍 인코딩 기준 워드피스당 값이므로 단어당 퍼플렉서티와 비교해서는 안 된다.

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

[2] K80, K40, M40, P100에 대해 각각 2.8, 3.7, 6.0, 9.5 TFLOPS를 사용했다.

표 3의 행 (A)에서는 3.2.2절에 기술한 대로 계산량을 일정하게 유지한 채 어텐션 헤드 수와 어텐션 키·값 차원을 바꿨다. 단일 헤드 어텐션은 최적 설정보다 0.9 BLEU 나쁘고, 헤드가 너무 많아도 품질이 떨어진다.

행 (B)에서는 어텐션 키 크기 d_k를 줄이면 모델 품질이 손상됨을 본다. 호환성 판별이 쉽지 않으며 내적보다 정교한 호환 함수가 유리할 수 있음을 시사한다. 행 (C)와 (D)에서는 기대대로 더 큰 모델이 더 낫고, 드롭아웃이 과적합 방지에 매우 도움이 됨을 확인했다. 행 (E)에서는 사인 위치 인코딩을 학습된 위치 임베딩 [9]으로 바꿨고, 베이스 모델과 거의 같은 결과를 얻었다.

6.3. 영어 구문 분석

트랜스포머가 다른 과제로 일반화되는지 보기 위해 영어 구문 분석 실험을 수행했다. 이 과제는 출력이 강한 구조적 제약을 받고 입력보다 훨씬 길다는 점에서 난해하다. 또한 RNN 시퀀스-투-시퀀스 모델은 소량 데이터 체제에서 최첨단에 도달하지 못했다 [37].

펜 트리뱅크(Penn Treebank)의 WSJ 부분 [25]에서 약 40K 학습 문장으로 $d_{model} = 1024$인 4층 트랜스포머를 학습했다. 더 큰 고신뢰·BerkleyParser 말뭉치(약 17M 문장)를 쓴 준지도 설정도 학습했다 [37]. WSJ 전용은 16K 토큰 어휘, 준지도는 32K 토큰 어휘를 썼다.

섹션 22 개발 집합에서 드롭아웃(어텐션·잔차, 5.4절), 학습률, 빔 크기만 소수 실험으로 고르고 나머지 매개변수는 영어-독일어 베이스 번역 모델과 동일하게 두었다. 추론 시 최대 출력 길이는 입력 길이 + 300으로 늘렸다. WSJ 전용·준지도 모두 빔 크기 21, $\alpha = 0.3$을 썼다.

표 4: 트랜스포머는 영어 구문 분석에도 잘 일반화된다 (WSJ Section 23 결과).

Table 4: The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ)

Parser	Training	WSJ 23 F1
Vinyals & Kaiser el al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser el al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

표 4의 결과는 과제 특화 튜닝이 없음에도 모델이 의외로 잘 동작하며, 순환 신경망 문법(Recurrent Neural Network Grammar) [8]을 제외한 이전 보고 모델보다 나은 결과를 낸다는 것을 보여 준다. RNN 시퀀스-투-시퀀스 모델 [37]과 달리, 트랜스포머는 40K 문장의 WSJ 학습 집합만으로 학습해도 BerkeleyParser [29]를 능가한다.

7. 결론

본 연구는 순환 층 대신 다중 헤드 자기-어텐션으로 인코더-디코더에서 가장 흔히 쓰이는 순환 층을 대체한, 전적으로 어텐션에 기반한 최초의 순열 변환 모델인 트랜스포머를 제시했다.

번역 과제에서 트랜스포머는 순환·합성곱 층 기반 아키텍처보다 훨씬 빠르게 학습할 수 있다. WMT 2014 영어-독일어와 영어-프랑스어 번역 모두에서 새로운 최첨단을 달성했으며, 전자에서는 이전에 보고된 모든 앙상블까지 포함해 최고 모델을 앞질렀다.

어텐션 기반 모델의 앞날에 기대하며, 다른 과제에 적용할 계획이다. 텍스트 이외의 입·출력 양식을 다루는 문제로 트랜스포머를 확장하고, 이미지·오디오·비디오처럼 큰 입출력을 효율적으로 다루기 위해 국소·제한 어텐션 메커니즘을 조사할 예정이다. 생성을 덜 순차적으로 만드는 것도 연구 목표이다.

모델 학습·평가에 사용한 코드는 https://github.com/tensorflow/tensor2tensor에서 이용할 수 있다.

8. 사사

Nal Kalchbrenner와 Stephan Gouws에게 유익한 의견, 수정, 영감에 감사드린다.

9. References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. arXiv preprint arXiv:1607.06450, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. CoRR, abs/1409.0473, 2014.

[3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. CoRR, abs/1703.03906, 2017.

[4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. arXiv preprint arXiv:1601.06733, 2016.

[5] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. CoRR, abs/1406.1078, 2014.

[6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. arXiv preprint arXiv:1610.02357, 2016.

[7] Junyoung Chung, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. CoRR, abs/1412.3555, 2014.

[8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In Proc. of NAACL, 2016.

[9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. arXiv preprint arXiv:1705.03122v2, 2017.

[10] Alex Graves. Generating sequences with recurrent neural networks. arXiv preprint arXiv:1308. 0850, 2013.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 770–778, 2016.

[12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.

[13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. Neural computation, 9 (8):1735–1780, 1997.

[14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing, pages 832–841. ACL, August 2009.

[15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. arXiv preprint arXiv:1602.02410, 2016.

[16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In Advances in Neural Information Processing Systems, (NIPS), 2016.

[17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In International Conference on Learning Representations (ICLR), 2016.

[18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. arXiv preprint arXiv:1610.10099v2, 2017.

[19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In International Conference on Learning Representations, 2017.

[20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In ICLR, 2015.

[21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. arXiv preprint arXiv:1703.10722, 2017.

[22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. arXiv preprint arXiv:1703.03130, 2017.

[23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Lukasz Kaiser. Multi-task sequence to sequence learning. arXiv preprint arXiv:1511.06114, 2015.

[24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. arXiv preprint arXiv:1508.04025, 2015.

[25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. Computational linguistics, 19(2):313–330, 1993.

[26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In Proceedings of the Human Language Technology Conference of the NAACL, Main Conference, pages 152–159. ACL, June 2006.

[27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In Empirical Methods in Natural Language Processing, 2016.

[28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. arXiv preprint arXiv:1705.04304, 2017.

[29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL, pages 433–440. ACL, July 2006.

[30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. arXiv preprint arXiv:1608.05859, 2016.

[31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909, 2015.

[32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv preprint arXiv:1701.06538, 2017.

[33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(1):1929–1958, 2014.

[34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, Advances in Neural Information Processing Systems 28, pages 2440–2448. Curran Associates, Inc., 2015.

[35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In Advances in Neural Information Processing Systems, pages 3104–3112, 2014.

[36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. CoRR, abs/1512.00567, 2015.

[37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In Advances in Neural Information Processing Systems, 2015.

[38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. arXiv preprint arXiv:1609.08144, 2016.

[39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. CoRR, abs/1606.04199, 2016.

[40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift -reduce constituent parsing. In Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers), pages 434–443. ACL, August 2013.

10. 어텐션 시각화 (부록)

아래는 원문 PDF의 부록에 실린 그림 3–5에 대한 한국어 캡션이다. 본 저장소에서 추출된 임베디드 래스터 이미지는 그림 1–2(모델 아키텍처·어텐션 다이어그램)에 해당하며, 부록의 세부 시각화는 원문 PDF를 참고하면 된다.

그림 3: 인코더 자기-어텐션의 6층 중 5층에서, 어텐션 메커니즘이 장거리 의존성을 따라가는 예이다. 많은 어텐션 헤드가 동사 making'의 먼 의존성을 주목하며, making...more difficult' 구를 완성한다. 여기서는 making' 단어에 대한 어텐션만 보였다. 색은 서로 다른 헤드를 나타낸다. 컬러로 보는 것이 좋다.

그림 4: 6층 중 5층에 있는 두 어텐션 헤드로, 대명사 소결(anaphora) 해석에 관여하는 것으로 보인다. 위: 헤드 5의 전체 어텐션. 아래: its' 단어만에 대한 헤드 5와 6의 어텐션. 이 단어에 대해서는 어텐션이 매우 날카롭다.

그림 5: 많은 어텐션 헤드가 문장 구조와 관련된 것처럼 보이는 행동을 보인다. 위는 6층 중 5층의 인코더 자기-어텐션에서 서로 다른 두 헤드의 예이다. 헤드들은 분명히 서로 다른 과제를 학습했다.